SMARTBRIDGE

Let's Bridge the Gap

Date:	4 July 2026
Team ID:	XXXXXX
Project Name:	Credit Card Approval Prediction System
Maximum Marks:	3 Marks

Project Executable Files

SKILL WALLET — A SmartBridge Product

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all application modules and asset files)	**Yes**
2	README / Setup Guide (detailed installation guidelines)	**Yes**
3	requirements.txt (Python dependency files list)	**Yes**
4	Database schema / Seed CSV datasets configuration profiles	**Yes**
5	Environment configuration file (.env.example configuration schema)	**Yes**
6	Deployed application production URL (if hosted)	**Yes**
7	Executable binary / Serialized model serialization dump dumps (.pkl)	**Yes**
8	Dockerfile / Containerization deployment configurations	**NA**
9	Test evaluation routines, evaluation script validation files	**Yes**
10	Demo video walkthrough presentation submission	**Yes**

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

```
credit-card-approval-system/
│
├── data/
│   ├── clean_application_records.csv   # Preprocessed demographic data
│   └── clean_credit_records.csv        # Balanced financial status records
│
├── models/
│   ├── xgboost_credit_model.pkl        # Serialized trained model weights
│   └── scaler_transform.pkl            # Pre-configured input feature bounds
│
├── src/
│   ├── app.py                          # Flask application core endpoint
│   ├── preprocessing.py                 # Feature encoding & SMOTE balance scripts
│   └── inference.py                    # Real-time risk probability model service
│
├── templates/
│   ├── index.html                      # Interactive input form interface
│   └── result.html                     # Approval / Rejection dashboard view
│
├── tests/
│   └── test_pipeline.py                # Automated system evaluation assertions
│
├── .env.example                        # Security variable structures
├── README.md                           # Setup and configuration manual
└── requirements.txt                    # Framework dependency modules list
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	https://credit-card-approval-evaluator.render.com
Repository Link	https://github.com/khasvi-reddy/credit-card-approval-system
Hosting Platform Provider	Render Cloud / Streamlit Cloud Platform
Login Credentials (Demo)	No authentication required for basic public screening tier evaluation.
Demo Video Link	https://youtube.com/watch?v=demo-credit-scoring-pipeline

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Pre-requisites:
Python 3.9+ environment, pip dependency manager, Git version control system, and 4GB+ available local hardware memory bounds.

Step	Execution Instruction Command
Step 1	Clone the project repository workspace from version control: `git clone https://github.com/khasvi-reddy/credit-card-approval-system.git`
Step 2	Navigate to root workspace directory and install all library framework dependencies: `pip install -r requirements.txt`
Step 3	Configure internal environmental schemas variables from example profiles: `cp .env.example .env`
Step 4	Execute testing suites to assert localized pipeline integrity checks: `python -m pytest tests/`
Step 5	Launch the production web interface application local server router daemon: `python src/app.py`
Step 6	Launch a web browser instance and navigate to active local hosting client gateway: http://localhost:5000 or http://127.0.0.1:5000

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	High inference latency during the initial server cold-start routine on free hosting tiers.	Keep endpoint container warmed up via programmatic validation requests intervals. (Status: Managed)
2	Invalid alpha-character string entry into numerical credit boundary limits throws parsing exception.	Enforced strict HTML5 frontend type validation restrictions to prevent non-numeric injection streams. (Status: Resolved)
3	Occasional model out-of-bounds warning on highly anomalous customer asset inputs.	Capped outlier input maximum thresholds programmatically inside the preprocessing layer module. (Status: Stable)